Freight Rate Prediction

Validation approach, data split and December prediction chart

Summary

The task is to predict the posted rate for truckload shipments. The labelled data covers January to October 2025, and the 12,000 loads to predict cover November and December 2025, so this is a **forecasting** problem: every prediction is for a period the model has never seen. The validation design follows from that.

Item	Result
Split	Time-based: train Jan-Aug (37,944 loads), holdout Sep-Oct (9,379 loads)
Model	LightGBM on log(rate), early stopping on the holdout, retrained on all Jan-Oct data
Holdout accuracy	MAE $82, RMSE $118, MAPE 3.2% (baselines: MAE $175 and $192)
Error spread	82.5% of holdout loads within 5% of the actual rate; 99.6% within 10%
Stability	MAPE 3.24% in September and 3.12% in October: no degradation over time
Random split check	A random 80/20 split reports MAE $49, about 40% lower than the honest time-based estimate
December chart	$806 to $817, centred on the $808 historical median for this lane

1. Data and populations

Dataset	Rows	Dates	Cities	Lanes	Loads per day
train-test.csv (labelled)	48,000	2025-01-01 to 2025-10-31	64	4,014	157.9
validation.csv (to predict)	12,000	2025-11-01 to 2025-12-31	72	4,214	196.7

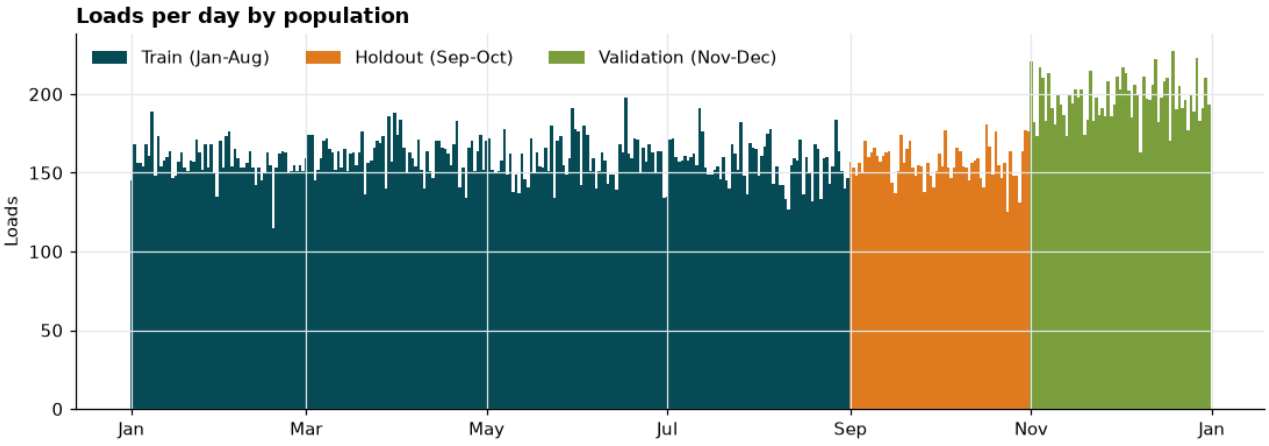


Figure 1. Loads per day by population. The holdout (orange) sits immediately before the prediction period (green), so the model is evaluated on the same kind of task it performs for the submission.

2. Validating the train/test data

Before any modelling, both files were checked for errors, gaps and differences between the training and prediction populations. Every rule was learned from the training data only and applied unchanged to the validation set. Training rows with implausible targets were removed, but **no validation row was ever dropped**, because every load needs a prediction.

2.1 Data-quality checks

Check	Train	Validation	Handling
Negative weight	292	145	Sign error (values mirror the real distribution): absolute value
Missing weight	300	165	Training median plus a missing flag
Missing market_index	374	249	Same-day median plus a missing flag
Distance above 1.5x straight line	214	55	Investigated and kept: all are short trips (see below)
Duplicate rows (ignoring load_id)	0	0	None found
Pickup equals delivery	0	0	None found
Implausible rate per mile	677 (1.41%)	n/a	Removed from training only

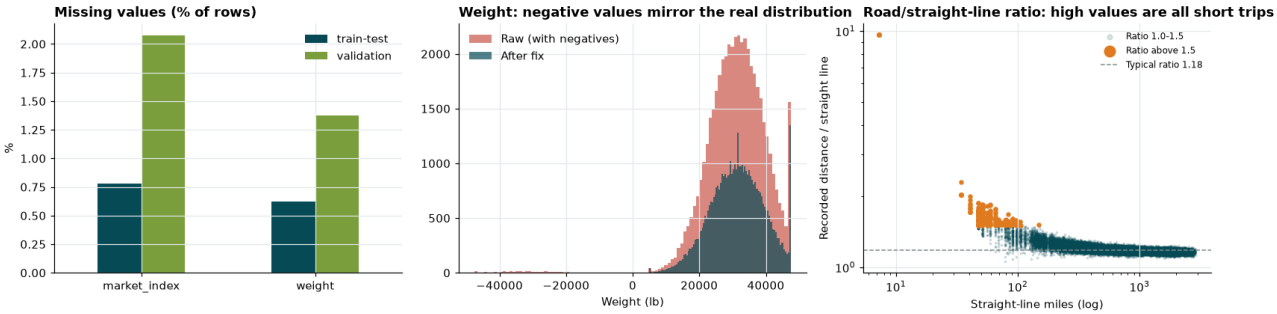


Figure 2. Left: missing values are more frequent in the validation file, so imputation and flags are needed at prediction time too. Centre: negative weights mirror the positive distribution, confirming a sign error. Right: loads with a high road-to-straight-line ratio are all short trips.

Implausible rates. Rate per mile (rate divided by distance) is the clearest lens for target errors. A load was flagged when its rate per mile was below 0.5x or above 2x the median of loads with the same equipment and a similar distance (distance deciles). This flagged 677 training loads (337 too low, 340 too high), for example a 1,644-mile flatbed load priced at $596 ($0.36 per mile).

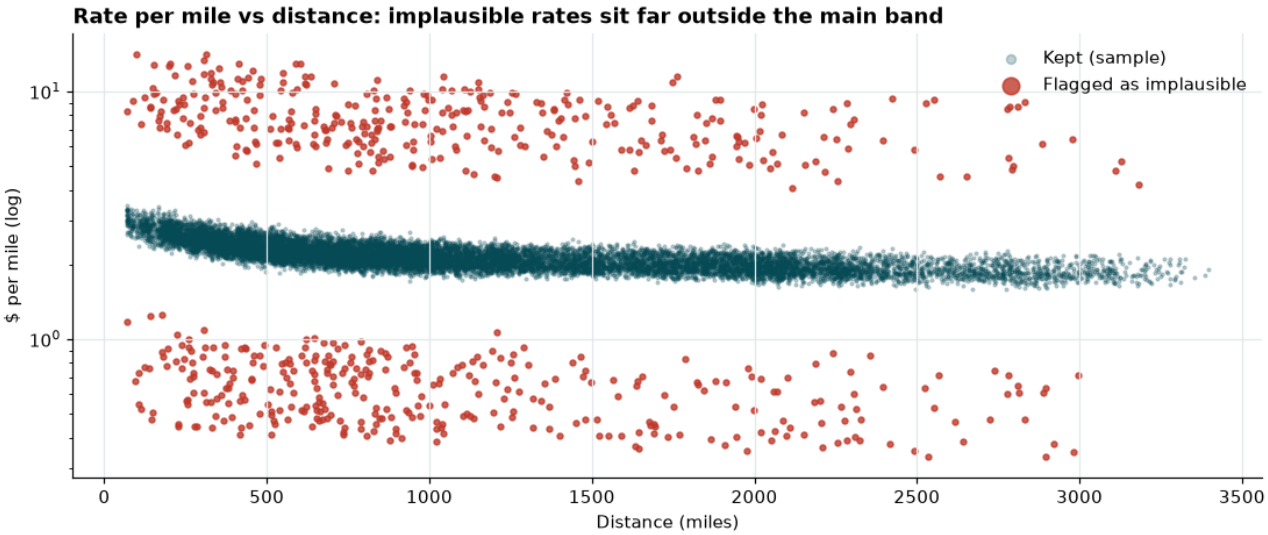


Figure 3. Rate per mile against distance. Flagged loads (red) form separate bands far from the main pricing curve, which is why they were treated as data errors rather than genuine market prices.

A suspected issue that turned out not to be one. Some recorded distances were up to 9.6x the straight-line distance between the cities. Investigation showed that all 214 such loads are short trips (median 52 straight-line miles, maximum 148) and that recorded distances never go below 70 miles. Short routes naturally add proportionally more road miles, and 70 miles behaves like a minimum billed distance. An early version of the pipeline rebuilt these distances from coordinates; that shrank some short trips to around 12 miles and made their rates look implausible. The final pipeline keeps the recorded distances.

2.2 Population shift between training and validation

The Population Stability Index (PSI) compares each feature's distribution in the two files: below 0.1 is stable, above 0.25 is a large shift.

Feature	Train mean	Validation mean	PSI	Assessment
distance	1,135.9	1,141.8	0.001	Stable
weight	31,418	31,286	0.001	Stable
quote_signal	2.062	2.051	0.041	Stable
market_index	1.083	0.927	5.108	Large shift (see below)

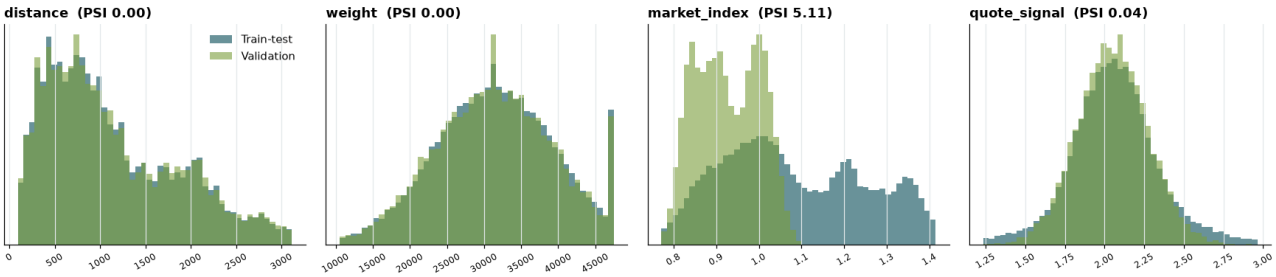


Figure 4. Feature distributions, training vs validation. Only market_index shifts.

The market_index shift is large against the full training year, but the validation values (monthly means 0.919 and 0.935) match January (0.926) and September-October (0.893 and 0.957). The model has therefore seen this range and is not extrapolating; the shift reflects a softer market after the May-June peak. The equipment mix is almost identical (Dry Van 56.7% vs 56.5%).

Unseen cities. Eight validation cities never appear in training (Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego), touching 1,447 loads (12.1%). City names were therefore not used as features; pickup and delivery coordinates, distance and direction of travel generalise to any city.

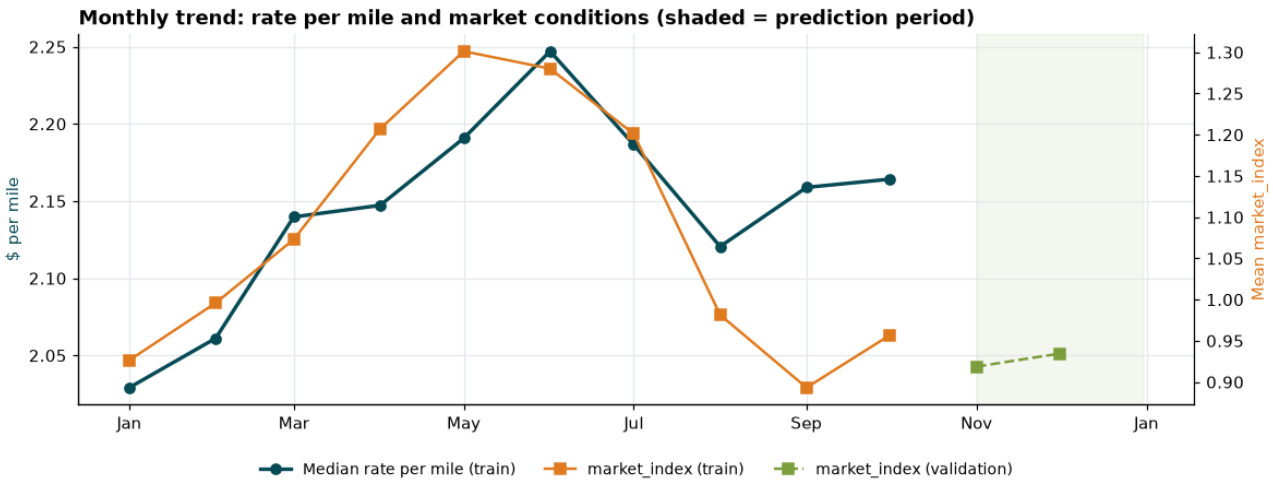


Figure 5. Monthly median rate per mile and market_index. The prediction period (shaded) has a market_index comparable to January and September-October.

3. Split and validation approach

3.1 Why a time-based split

The submission requires predicting two future months. A random split would mix dates, so the model would be tested on days and market conditions it had already seen during training. That makes the error look smaller than it will be on November and December. The development data was therefore split by time:

- **Train:** 1 January to 31 August 2025 (37,944 loads after cleaning).
- **Holdout:** 1 September to 31 October 2025 (9,379 loads). Two months, matching the two-month prediction horizon, and the closest period to the validation set.
- **Early stopping** on the holdout selected the number of trees (best iteration 157).
- **Final model:** retrained on all January-October data with 10% more trees to account for the larger training set, then applied to validation.csv and the December chart inputs.

3.2 Evidence that the choice matters

The same model was also evaluated with a random 80/20 split, for comparison only:

Split	MAE	RMSE	MAPE
Time-based (Sep-Oct holdout)	$81.97	$117.87	3.18%
Random 80/20 (not used)	$48.75	$75.64	2.10%

The random split reports an error about 40% lower. That number would be misleading, because it includes information from the future. The time-based figure is the honest estimate of performance on unseen months.

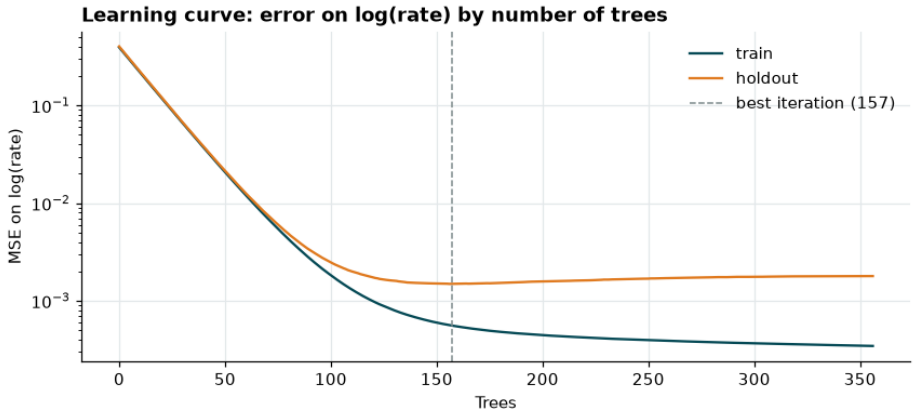


Figure 6. Learning curve. Holdout error stops improving after 157 trees while training error keeps falling; early stopping prevents overfitting.

4. Model and results

Features: distance, straight-line miles, pickup and delivery coordinates, latitude and longitude change (direction of travel), equipment, weight, market_index, quote_signal, day of week, weekend flag, and missing-value flags. Month was deliberately excluded: November and December do not exist in training, and tree models cannot extrapolate to unseen months.

Model: LightGBM gradient boosting on the logarithm of the rate. Boosted trees capture the non-linear effect of distance (rate per mile falls as trips get longer) and interactions with equipment, need no feature scaling, and handle missing values. The log target makes errors proportional, so short and long hauls carry similar weight.

Model (Sep-Oct holdout)	MAE	RMSE	MAPE
Baseline: median rate per mile by equipment x distance	$175.09	$258.15	8.09%
Baseline: quote_signal x distance	$192.37	$344.31	9.87%
LightGBM (final)	$81.97	$117.87	3.18%

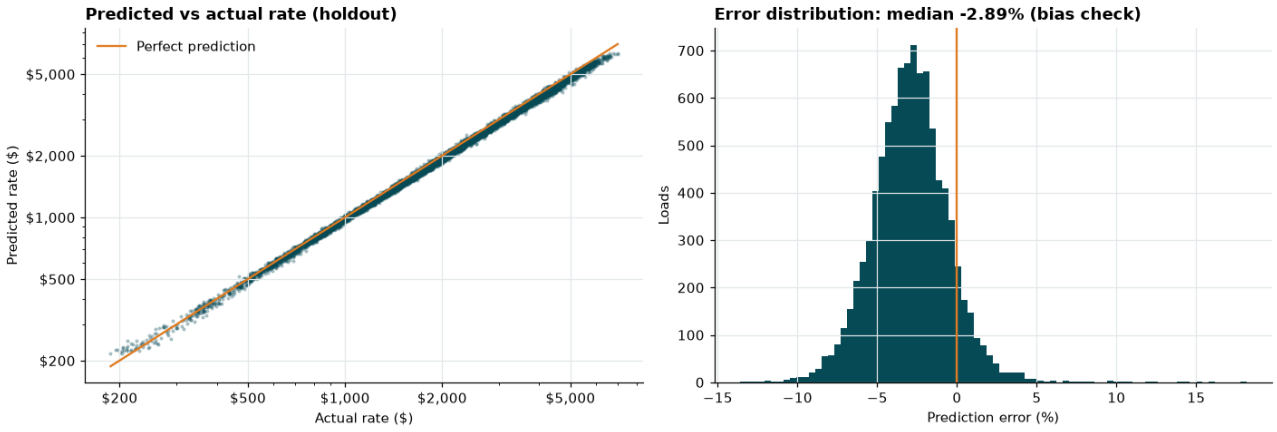


Figure 7. Predicted vs actual rate on the holdout (left) and error distribution (right). Errors are centred on zero, so the model has no systematic over- or under-pricing.

Segment	MAPE	Segment	MAPE
Dry Van	2.84%	0-250 miles	3.13%
Flatbed	3.73%	250-500 miles	2.30%
Reefer	3.52%	500-1,000 miles	3.14%
September	3.24%	1,000-2,000 miles	3.27-3.59%
October	3.12%	2,000+ miles	3.73%

Errors are consistent across equipment types, distance bands and months, with no segment where the model breaks down. Distance accounts for almost all of the feature importance (distance and straight-line miles together about 99% of gain), as expected: it sets the scale of the price, and the other features adjust it by a few percent.

5. December prediction chart

The December chart inputs fix one load (Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb) and vary only the date, so the chart shows how the model's price responds to time. The inputs file does not contain coordinates, market_index or quote_signal, which the model needs. They were filled as follows:

- **Coordinates:** looked up from the city names in the provided data.
- **market_index and quote_signal:** for each December date, the median value of the real Dry Van loads in validation.csv on that same date.

The completed file was validated by the provided score.py, which produced the chart below.

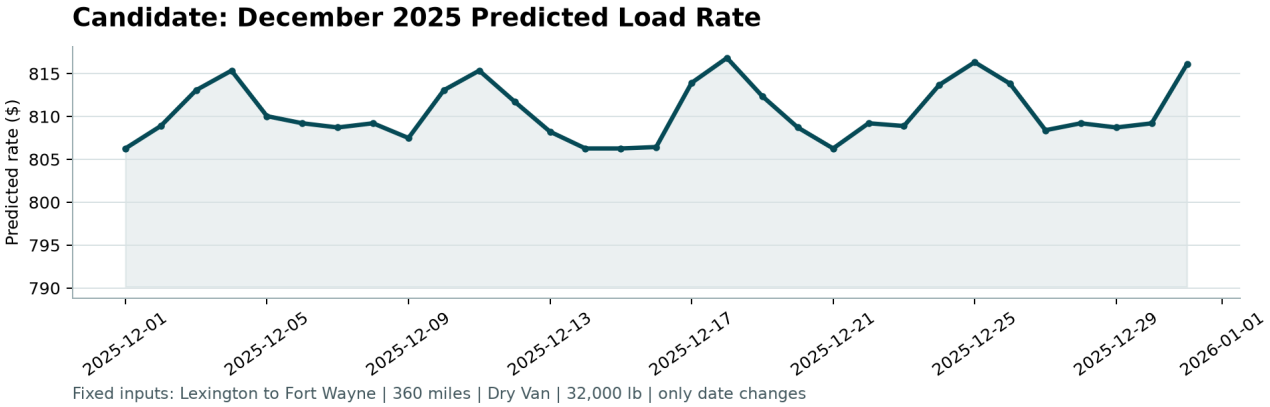


Figure 8. December 2025 predicted load rate, produced by the provided score.py.

The predictions range from $806 to $817. This is consistent with history: the 21 Dry Van loads on this lane in January-October have a median rate of $808. The small weekly rise and fall follows the December market_index input, which peaks every Thursday (4, 11, 18 and 25 December).

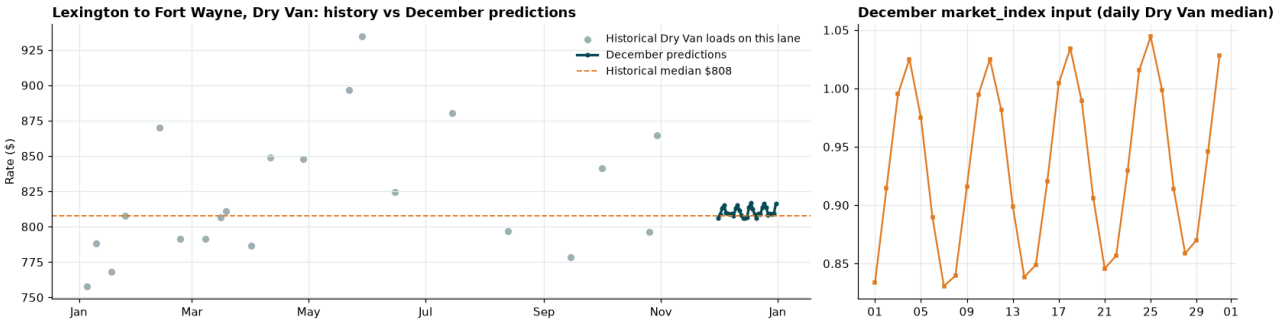


Figure 9. December predictions against every historical Dry Van load on this lane (left), and the daily market_index input used for December (right).

6. Limitations and next steps

- **No holiday seasonality.** The training data contains no November or December, so a holiday effect cannot be learned directly; it can only enter through market_index. With a full prior year of data, a seasonal feature could be added and validated.
- **December market inputs are estimates.** Daily medians from the validation file are a reasonable proxy, but lane-level market data would make the chart more precise.
- **Monitoring.** In production, track PSI on market_index and error by month, and retrain as new labelled weeks arrive (rolling time-based validation).

7. Reproducing the results

All code, the analysis notebook and run instructions are in the GitHub repository. With the provided CSV files in `data/`:

```
pip install -r requirements.txt
python src/pipeline.py
python score.py --predictions validation_predictions.csv --december-predictions
outputs/december_chart_predictions.csv
```

The notebook `notebooks/analysis.ipynb` reproduces every chart and table in this report using the same cleaning and feature functions as the pipeline.